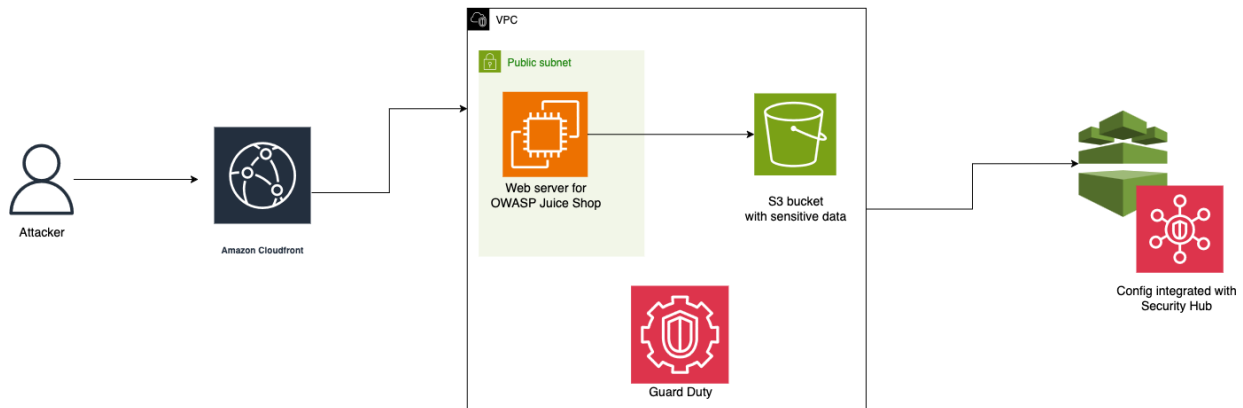


OFFENSE-TO-DEFENSE

(THREAT DETECTION USING GUARD-DUTY)



PREPARATION

Created and deployed a CloudFormation template for reproducible adversary emulation in AWS, simulating an end-to-end attack → detect → remediate scenario.

```
7
8 Parameters:
9   KeyName:
10    Type: AWS::EC2::KeyPair::KeyName
11    Description: EC2 KeyPair for SSH (optional; required to SSH)
12    Default: ayamtired
13   InstanceType:
14    Type: String
15    Default: t3.micro
16    Description: Instance type for ASG instances
17   SSHLocation:
18    Type: String
19    Default: 0.0.0.0/0
20    Description: Source CIDR for SSH (for testing only; lock down in production)
21
```

The lab involved spinning up Juice Shop, simulating credential theft, validating detection and hardening controls.

Template Preparation

From the CloudFormation Stacks page, selected Create Stack, with new resources (standard).

On the Create Stack page, under Specify a template, chose upload a template file, selected the yaml file from local storage, and confirmed the file was uploaded.

The screenshot shows the 'Create stack' page in the AWS console, specifically the 'Specify template' section. The page is divided into two main parts: 'Prerequisite - Prepare template' and 'Specify template'. In the 'Specify template' section, the 'Template source' is set to 'Upload a template file'. Below this, the 'Upload a template file' section shows a file named 'Guard-Duty.yaml' selected for upload. The file is identified as a 'JSON or YAML formatted file'.

Create stack

Prerequisite - Prepare template

You can also create a template by scanning your existing resources in the [IaC generator](#).

Prepare template

Every stack is based on a template. A template is a JSON or YAML file that contains configuration information about the AWS resources you want to include in the stack.

☒ Choose an existing template
Upload or choose an existing template.

☐ Build from Infrastructure Composer
Create a template using a visual builder.

Specify template Info

This [GitHub repository](#) contains sample CloudFormation templates that can help you get started on new infrastructure projects. [Learn more](#)

Template source

Selecting a template generates an Amazon S3 URL, where it will be stored. A template is a JSON or YAML file that describes your stack's resources and properties.

☐ Amazon S3 URL
Provide an Amazon S3 URL to your template.

☒ Upload a template file
Upload your template directly to the console.

☐ Sync from Git
Sync a template from your Git repository.

Upload a template file

Guard-Duty.yaml

JSON or YAML formatted file

Verified the template selection and proceeded by clicking next.

Stack Details

On the Specify Stack Details page, entered a stack name.

Left parameters at their default values and selected next to complete it.

Next, On the Configure Stack Options page, checked the box acknowledging that CloudFormation might create IAM resources.

Chose Next to continue.

Review and Deployment

Reviewed the configuration on the Review and create page.

Submitted the stack for deployment.

The stack creation process took about 10 minutes to complete.

After successful creation, reviewed the Outputs tab.

Recorded the values for JuiceShopURL and TheSecureBucket for later use.

Create stack

Step 1: Specify template

Prerequisite - Prepare template

Template

Template is ready

Template

Template URL

<https://s3.eu-north-1.amazonaws.com/cf-templates-cldo3t4rstcn-eu-north-1/2025-09-25T102523.70520ca-Guard-Duty.yaml>

Stack description

Deploy OWASP Juice Shop (Docker) behind ALB, fronted by CloudFront. Includes VPC, subnets, IGW, route table, ALB, LaunchTemplate + AutoScaling, S3 bucket for "sensitive.txt" (instance uploads file on boot), GuardDuty detector, and an S3 VPC endpoint.

The screenshot shows the AWS CloudFormation console. At the top, there's a navigation bar with the AWS logo, a search bar, and account information (Europe (Stockholm), Account ID: 1630-8893-3322, UltimatelAM). Below the navigation bar, there's a blue banner with a message about introducing a new Control Catalog integration for AWS CloudFormation Hooks. The main content area shows the 'Stacks (1)' list on the left, with 'Ro-GuardDuty2' selected. The stack's status is 'CREATE_COMPLETE'. The right pane shows the 'Stack info' tab for 'Ro-GuardDuty2', displaying the stack ID, description, status, status reason, parent stack, and created time.

Stacks (1)

Filter by stack name: Filter status: Active

☒ View nested

Stacks

- Ro-GuardDuty2
2025-09-25 14:55:59 UTC+0000

Ro-GuardDuty2

Stack info | Events | Resources | Outputs | Parameters | Template | Change sets | Git sync

Overview

Stack ID
[arn:aws:cloudformation:eu-north-1:163088933322:stack/Ro-GuardDuty2/bf173c00-9a1f-11f0-83ff-0ea265cef27d](#)

Description
Deploy OWASP Juice Shop (Docker) behind ALB, fronted by CloudFront. Includes VPC, subnets, IGW, route table, ALB, LaunchTemplate + AutoScaling, S3 bucket for "sensitive.txt" (instance uploads file on boot), GuardDuty detector, and an S3 VPC endpoint.

Status

Detailed status
-

Status reason
-

Root stack
-

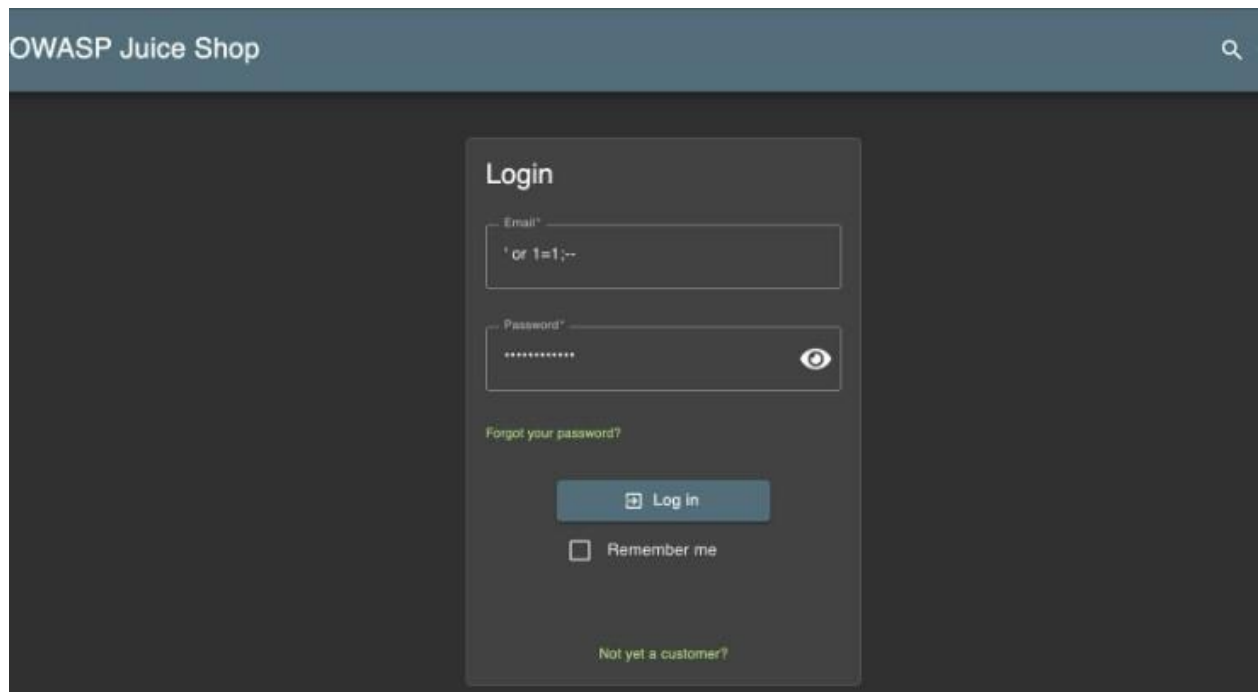
Parent stack
-

Created time
2025-09-25 14:55:59 UTC+0000

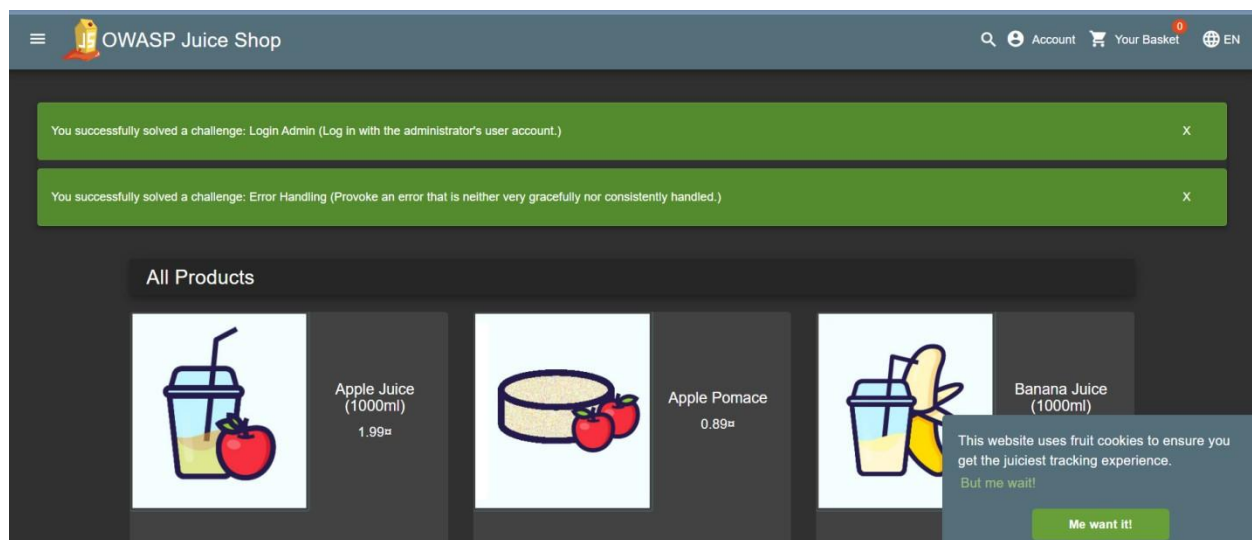
Next, accessed the deployed Juice Shop from the URL. The OWASP Juice Shop homepage loads. Dismissed the initial welcome dialog, and accepted the cookie notice when prompted.



Authenticated as an administrator using a login bypass technique.



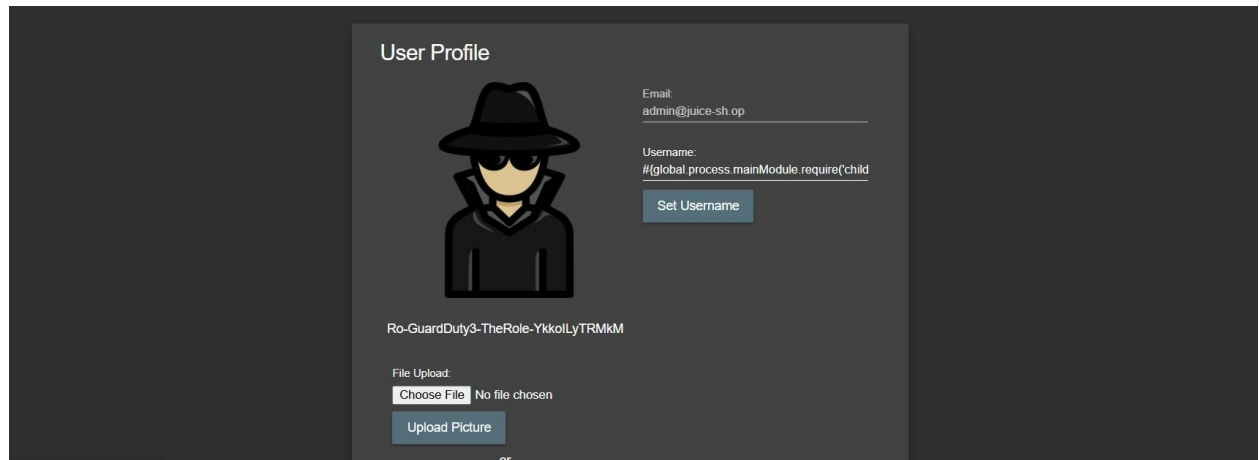
After submitting the credentials, confirmed that the sign-in succeeded.



GETTING AWS CREDENTIALS

From the admin account, open Account → User Profile. On the profile page, set the username using an input that triggers server-side command execution.

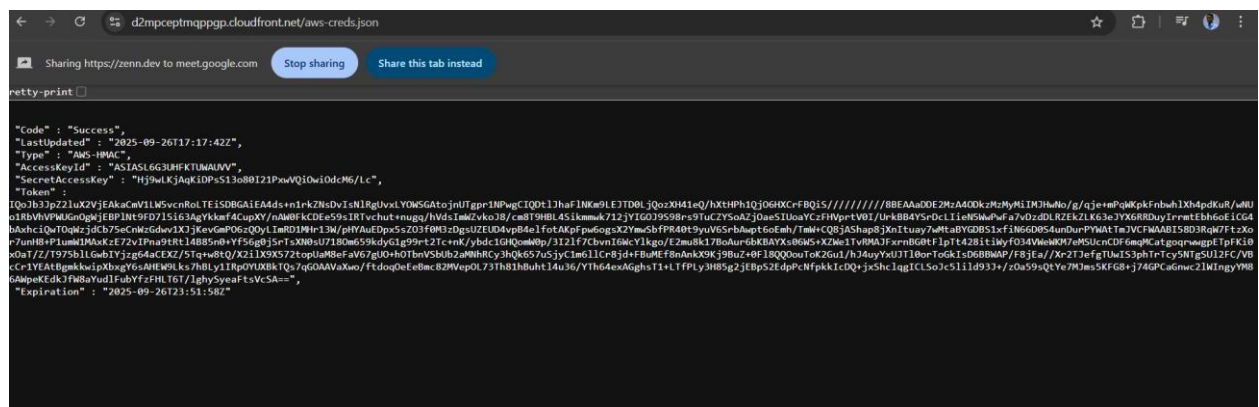
Verified that the page shows the command output beneath the username avatar, indicating the process/user identity was printed to the page.



The public directory aws-creds.json was then created. I accessed it by using the following URL attached to the Juice Shop homepage Url:

<JuiceShopURL>/aws-creds.json

The contents of the JSON file should look like this (the values will most likely be different): I copied the details for the AccessKeyId, SecretAccessKey, and Token. The three elements that make up the authentication information, AccessKeyId, SecretAccessKey, and Token are valid until the Expiration date and time. In Lehmann's terms, the attacker has now been successful in obtaining the web application's AWS authentication credentials.



RETRIEVING FILES IN

ro-guardduty3-thesebucket-uywxrq3buuj [Info](#)

[Objects](#) | [Properties](#) | [Permissions](#) | [Metrics](#) | [Management](#) | [Access Points](#)

Objects (1) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐

Name	Type	Last modified	Size	Storage class
☐ secret-information.txt	txt	September 26, 2025, 17:19:14 (UTC+00:00)	78.0 B	Standard

Open a terminal prompt on the machine for the lab, go through the AWS Configure process and then set the AWS credentials obtained as the environment variables

Next, set an environment variable for the S3 bucket name using the value shown in the CloudFormation stack outputs (TheSecureBucket value): set
AWS_SECURE_BUCKET=<TheSecureBucket value>

Next, list the objects in the bucket with:

```
aws s3 ls s3://%AWS_SECURE_BUCKET%
```

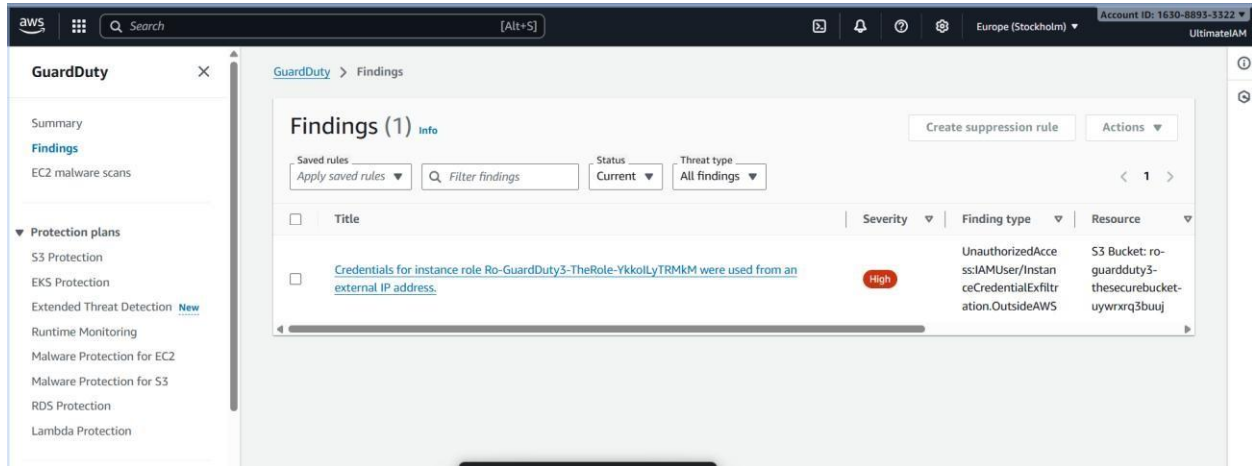
Next, download the `secret-information.txt` object to the current directory: aws

```
s3 cp s3://%AWS_SECURE_BUCKET%/secret-information.txt
```

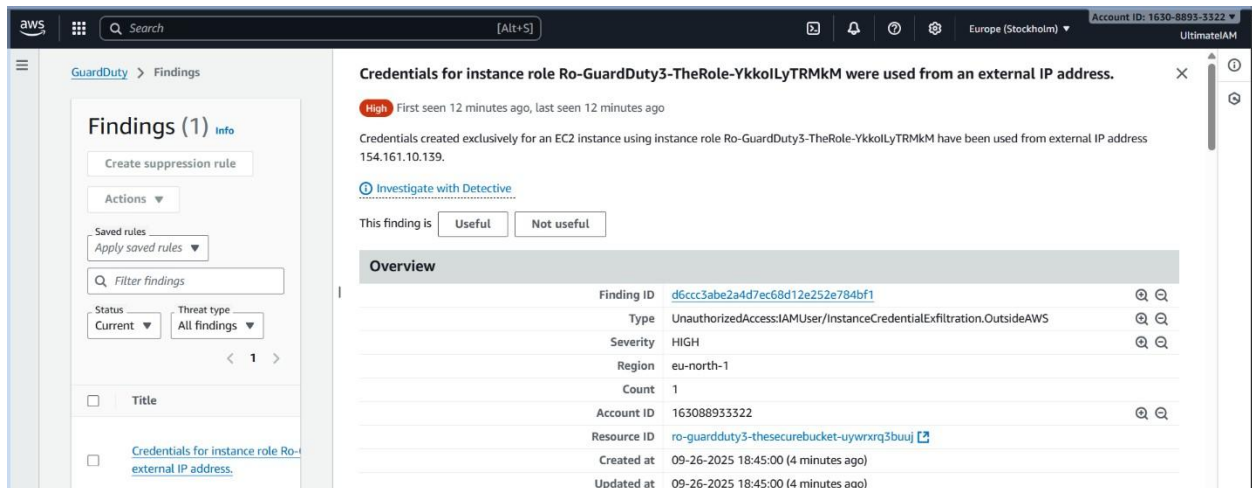
```
C:\Users\user>set AWS_ACCESS_KEY_ID=ASIASL6G3UHFKTUNAUUV
C:\Users\user>set AWS_SECRET_ACCESS_KEY=Hj9wLKjAqKiDPsS13o80I21PxwVQioWi0dcM6/Lc
C:\Users\user>set AWS_SESSION_TOKEN=IQaJb3JpZ2LuX2VjEAKaCmVlW5vcnRoLTEiSDBGAiEA4ds+n1rkZNsDvIsNlRgUvXLYOWSGAtojnUTgpr1NPwgCIQDtlJhaFLNkM9LEJTD8LjQozXH41eQ/hXtHPH1QjO6HXCrFBQis/////////8BEAAaDDE2MzA4ODkzMzMyMiIMJHwNo/g/qje+mPqMKpKfNbwhLXh4pdKuR/wNUco1RbVhVPWUGnOgWjEBPLnt9FD7L5i63AgYkkmf4CupXY/nAW8FkCDEe59sIRTVchut+nugq/hVdsInWZykoJ8/cm8T9HBL4S1kmmwk712jYIG0J9S98rs9TuCZYSoAZj0aeSIUoaYCYZFHVprtV8I/UrKBB4YSrDcLIeN5WwPwFa7vDzdDLRZEKZLK63eJYX6RRDuyIrrmtEb60eICG4gbAxbciQwTQwZjdCb75eCnWzGdwv1XJjKevGmP06zQ0yLImRD1MH+13W/pHYAuEDpx5sZ03f0M3zDgsUZEUd4vpB4eLfotAKpFpw6ogsX2YmwSbFPR40t9yuV6SrbAwpt6oEmh/TmW+CQ8jXShap8jXnItuay7wMtaBYGDBS1xfIN66D0S4unDurPYWAtTmJVCFWAABIS8D3RqW7FtzXozr7unH8+P1umM1MAxKzE72vIPna9tRtL4B85n0+Yf56g0jSrTsXN0sU7180m659kdyG1g99rt2Tc+nK/ybdc1GHQomW0p/3I2L7fCbnI6WcYlkgO/E2mu8k17BoAur6bKBAyxS06WS+XZWe1TVrMAJFxrnbG0tFlpTt428itiYf034VWwKMK7eMSUcnCDF6mqMCatgoqrwwgpETpFKi0ax0aT/Z/T975bLLGwbIYjzg64aCEXZ/5Tq+w8tQ/X2iLX9X572topUaM8eFaV67gUO+h0TbnVSBub2aMNHRCy3hQk657u5jyC1m6LLCr8jd+FBuMEf8nAnkX9Kj9BuZ+0FL8QQ0ouToK2Gu1/hJ4uyYxUJTL0orToGkIsD6BBWAP/F8jEa//Xr2TJefgTUwIS3phTrTcy5NTgSUL2FC/VBEcCr1YEAtBgmkkwipXbxyG6sAHEW9Lks7hBLy1IRpOYUXBkTQs7qG0AAVAxwo/ftdoqOeEeBmc82MvEPOL73Th81hBuhtL4u36/YTh64exAGghsT1+LTFPLy3H85g2jEBpS2EdpPcNfpkKIcDQ+jxShcLqgICLSoJcS1ild93J+/z0a59sQtYe7Mjns5KFG8+j74GPCaGnwc2LWingyYM8b6AWpeKEdkJfW8aYudLFubYfzFHLT6T/LghySyeaFtsVcSA==
C:\Users\user>set AWS_SECURE_BUCKET="ro-guardduty3-thesebucket-uywxrq3buuj"
C:\Users\user>aws s3 ls s3://%AWS_SECURE_BUCKET%
2025-09-26 17:19:14          78 secret-information.txt
C:\Users\user>aws s3 cp s3://%AWS_SECURE_BUCKET%/secret-information.txt .
download: s3://ro-guardduty3-thesebucket-uywxrq3buuj/secret-information.txt to .\secret-information.txt
C:\Users\user>
```

USING GUARDDUTY TO SPOT SUSPICIOUS BEHAVIOR

Navigate to the GuardDuty console in AWS. Check to see if the usage of stolen AWS credentials has been identified. It could take up to 10s of minutes to detect.



GuardDuty monitors communication packet flows documented in VPC Flow Logs and AWS actions documented in CloudTrail, and it identifies any unusual or suspicious activity. In this case, it was discovered that session credentials meant to be used inside an EC2 instance were being utilized from an external IP address or a different AWS account.



Depending on the extent of the records and whether the incident took place within the retention period, a skilled security administrator can then identify that credentials have been stolen and, by tracking the AWS operations carried out using those credentials, may find that sensitive files have been leaked from S3.

The following tables represent the data shown in the screenshots:

Resource affected

Resource role	TARGET
Resource type	S3Bucket
Access key ID	ASIASL6G3UHFKTUWUUV
Principal ID	AROASL6G3UHFPSWD7WY7-i-096625da80ea6cc73
User type	AssumedRole
Username	Ro-GuardDuty3-TheRole-YkkoLyTRMkM

Instance details

Instance ID	i-096625da80ea6cc73
Instance type	t3.micro
Instance state	running
Availability zone	eu-north-1a
Image ID	ami-01637463b2cbe7cb6
Image description	Canonical, Ubuntu, 20.04, amd64 focal image
Launch time	09-26-2025 17:16:56

IAM instance profile

ARN	arn:aws:iam:163088933322:instance-profile/Ro-GuardDuty3-TheInstanceProfile-USCR8J9t0DUU
ID	AIPASL6G3UHFJWTCPNOD7

Instance tags

aws:cloudformation:logical-id	TheAutoScalingGroup
aws:cloudformation:stack-name	Ro-GuardDuty3
aws:cloudformation:stack-id	arn:aws:cloudformation:eu-north-1:163088933322:stack/Ro-GuardDuty3/7aa51280-9afc-11f0-9988-0a7985bc5971
Name	Ro-GuardDuty3
aws:ec2:launchtemplate:version	1
aws:autoscaling:groupName	Ro-GuardDuty3-TheAutoScalingGroup-noMztZu6sew
aws:ec2:launchtemplate:id	lt-0c419efe4e1b6d88c

Affected resources

S3 buckets

Destination: ro-guardduty3-thesebucket-uywxrq3buuj

Name	ro-guardduty3-thesebucket-uywxrq3buuj
Type	Destination
ARN	arn:aws:s3::ro-guardduty3-thesebucket-uywxrq3buuj
Effective permission	NOT_PUBLIC
Created at	09-26-2025 17:16:24 UTC

Default server side encryption

Encryption type	AES256
-----------------	--------

Owner

ID	eab6aa8762268a00d723587b591aa6e541fd297fc38742c2e70ec6c9c17e0a8b
----	--

Tags

aws:cloudformation:stack-id	arn:aws:cloudformation:eu-north-1:163088933322:stack/Ro-GuardDuty3/7aa51280-9afc-11f0-9988-0a7985bc5971
aws:cloudformation:stack-name	Ro-GuardDuty3
aws:cloudformation:logical-id	TheSecureBucket

DETECTING MISCONFIGURATIONS WITH CONFIG AND SECURITY HUB

Enable AWS Config. In the AWS Management Console, open Config. Select Start Now.

On the Settings page, keep the defaults and click Next. On the Rules page, leave the defaults and click Next. On the Review page, choose Confirm.

Enable Security Hub & Best Practices

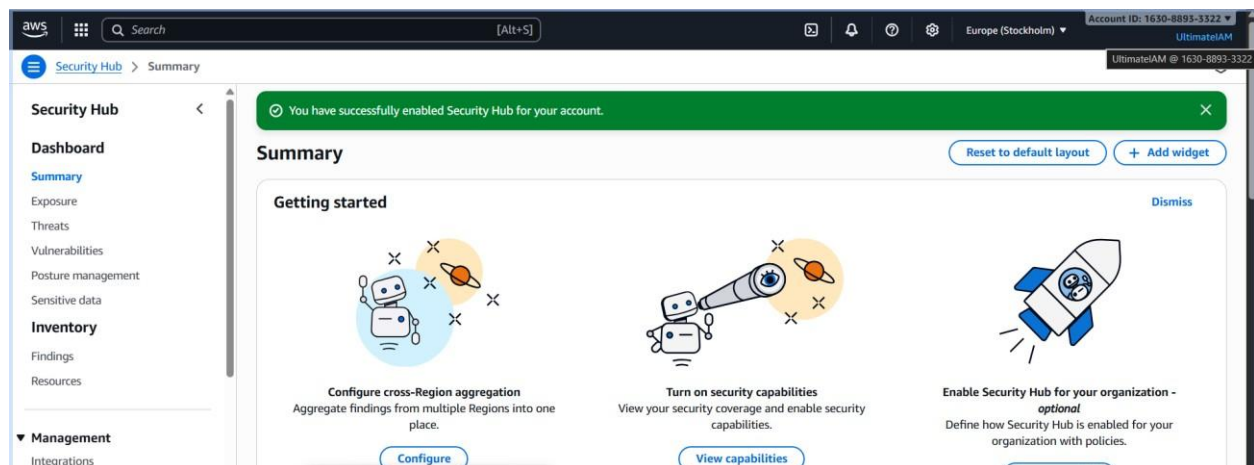
In the console, open Security Hub and click Start using.

On the Enable Security Hub page, choose Enable.

After enabling, you'll see a confirmation message. Click Go to Security Hub.

From the navigation pane, select Detection Engine → Security Hub CSPM.

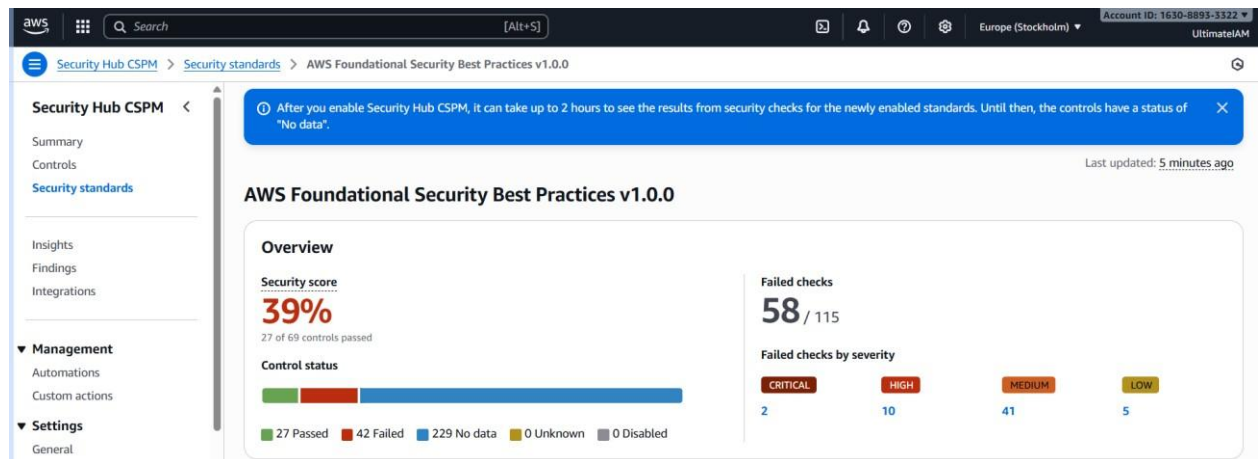
On the CSPM homepage, select Enable AWS Foundational Security Best Practices v1.0.0 and confirm.



If Security Hub CSPM was already enabled, go to Security Standards and enable AWS Foundational Security Best Practices v1.0.0 manually.

Check Results

Once the checks run, the Security Hub CSPM dashboard will show a security score and highlight failed controls.



Open the AWS Foundational Security Best Practices v1.0.0 report to view details.

Use the search filter control = FAILED to display only failed controls.

Switch to ID-level view by clicking the ID column.

Confirm that one detected issue matches the root cause of the attack:

EC2.8: EC2 instances should enforce Instance Metadata Service Version 2 (IMDSv2).

